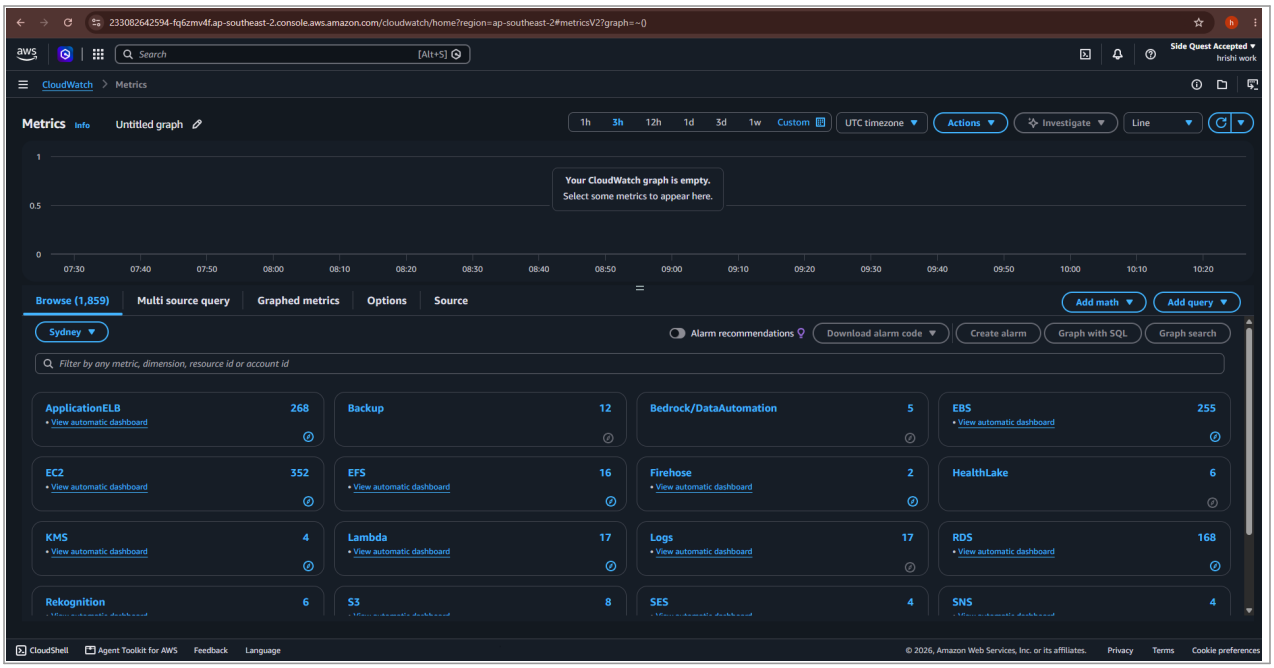
AWS CLOUDWATCH TASKS

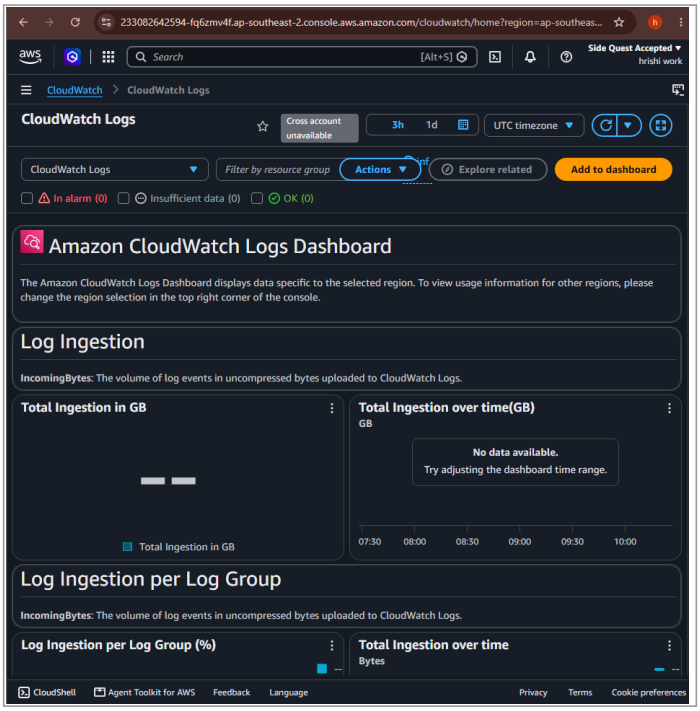
Submitted by Hrishinandan P

Task 1: Explore the CloudWatch Console

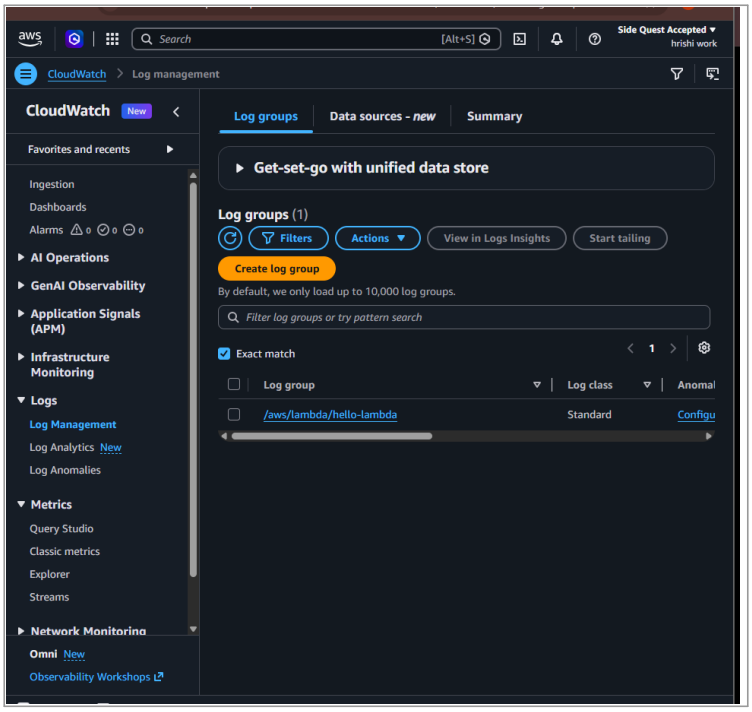
- Explored the CloudWatch console sections: Metrics, Logs, Dashboards and Application Signals.
- The Metrics page lists the AWS services publishing metrics, such as EC2, EBS, RDS, Lambda, S3, SNS and Logs.



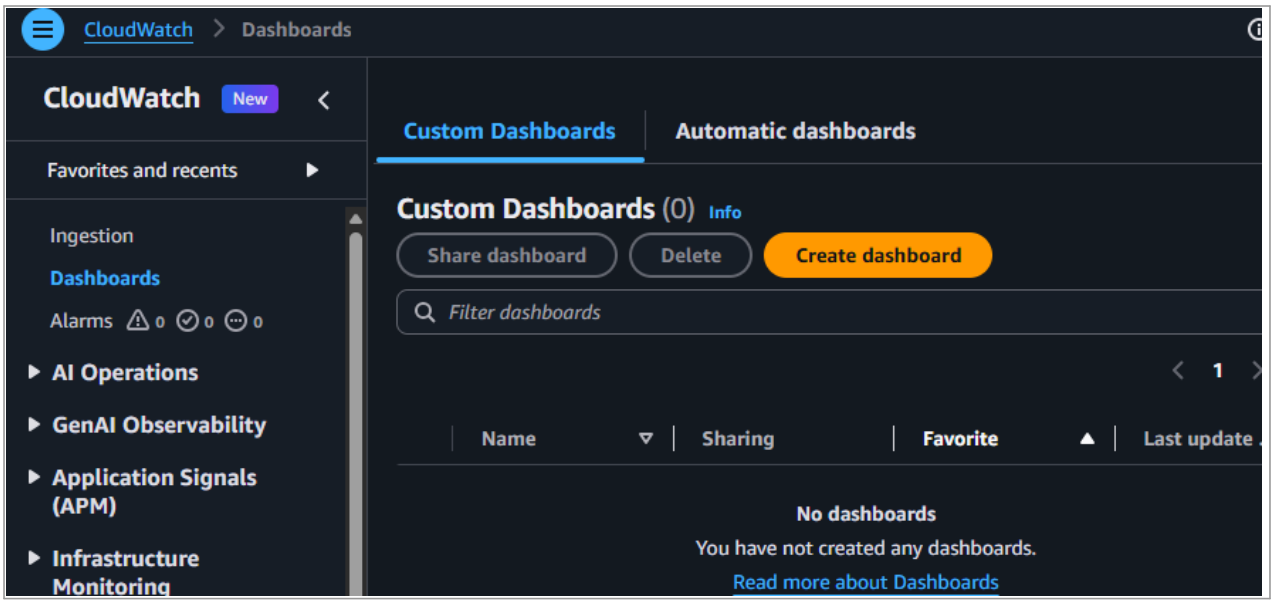
Opened CloudWatch Metrics and browsed the namespaces.



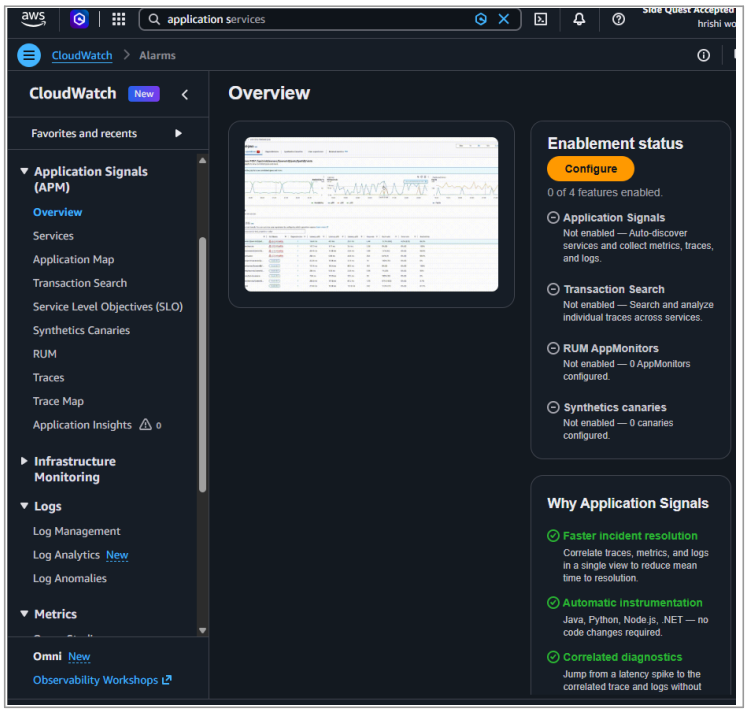
Opened the CloudWatch Logs dashboard.



Viewed Log Management, which shows the log group /aws/lambda/hello-lambda.



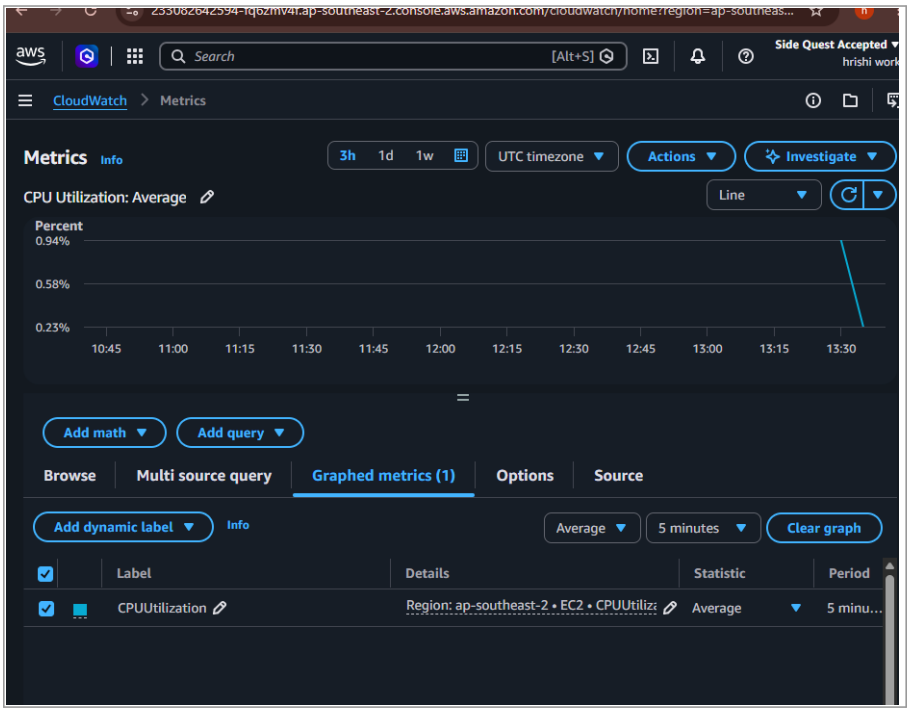
Opened the Dashboards page (no custom dashboards yet).



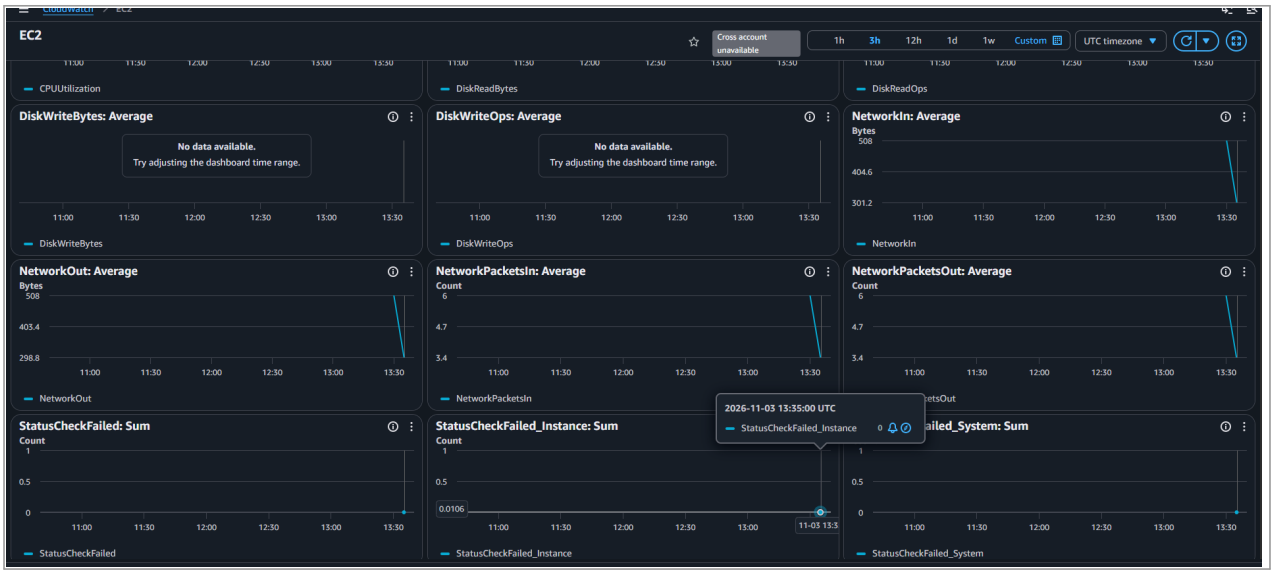
Opened the Application Signals overview.

Task 2: Monitor an EC2 Instance

- Monitored the EC2 instance's CPU Utilization in CloudWatch Metrics.
- Checked Network In/Out, Disk Read/Write and Status Check metrics on the EC2 dashboard. Status checks show 0 failures.



Graphed the EC2 CPUUtilization metric (Average, 5-minute period).



EC2 automatic dashboard showing Disk, Network and Status Check metrics.

Task 3: Install the CloudWatch Agent

- Installed the CloudWatch Agent on the EC2 instance and configured it to collect memory, disk and swap utilization.
- Started the agent and confirmed that memory and swap metrics appear in CloudWatch under the CWAgent namespace.

```
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :
  Running scriptlet: amazon-cloudwatch-agent-1.300073.2-1.amzn2023.x86_64
create group cwagent, result: 0
create user cwagent, result: 0

  Installing       : amazon-cloudwatch-agent-1.300073.2-1.amzn2023.x86_64
  Running scriptlet: amazon-cloudwatch-agent-1.300073.2-1.amzn2023.x86_64
  Verifying        : amazon-cloudwatch-agent-1.300073.2-1.amzn2023.x86_64

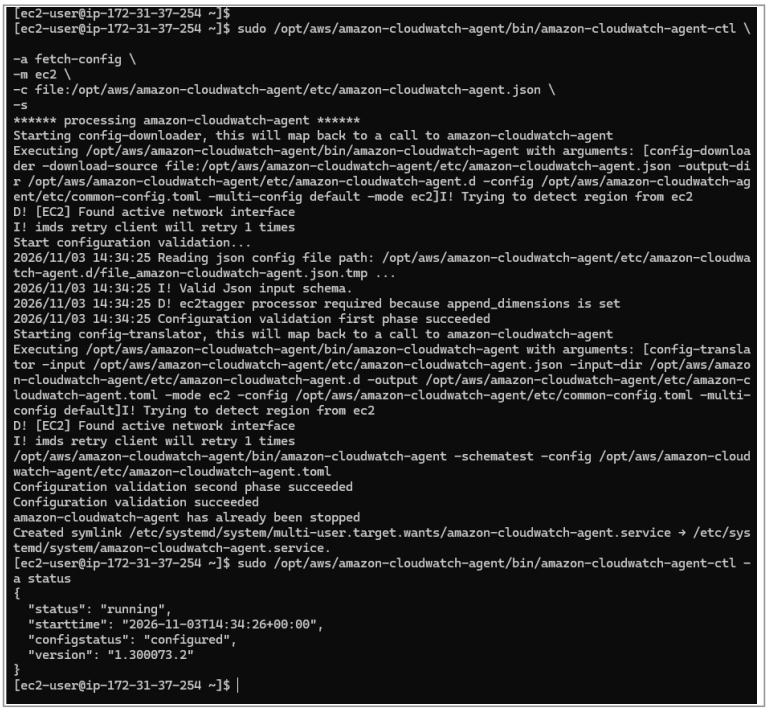
Installed:
  amazon-cloudwatch-agent-1.300073.2-1.amzn2023.x86_64

Complete!
[ec2-user@ip-172-31-37-254 ~]$ /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
{
  "status": "stopped",
  "starttime": "",
  "configstatus": "not configured",
  "version": "1.300073.2"
}
[ec2-user@ip-172-31-37-254 ~]$
```

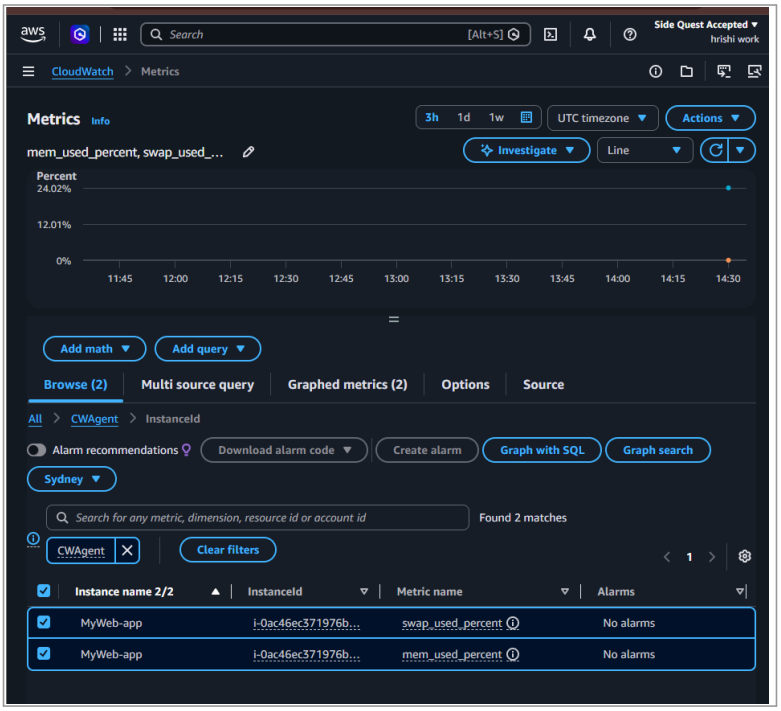
Installed amazon-cloudwatch-agent. Status: stopped, not configured.

```
{
  "agent": {
    "metrics_collection_interval": 60,
    "run_as_user": "root"
  },
  "metrics": {
    "namespace": "CWAgent",
    "append_dimensions": {
      "InstanceId": "${aws:InstanceId}"
    },
    "metrics_collected": {
      "mem": {
        "measurement": [
          "mem_used_percent"
        ],
        "metrics_collection_interval": 60
      },
      "disk": {
        "measurement": [
          "disk_used_percent"
        ],
        "resources": [
          "/"
        ],
        "ignore_file_system_types": [
          "sysfs",
          "devtmpfs",
          "tmpfs"
        ],
        "metrics_collection_interval": 60
      },
      "swap": {
        "measurement": [
          "swap_used_percent"
        ],
        "metrics_collection_interval": 60
      }
    }
  }
}
~
```

Agent JSON config for memory, disk and swap used percent.



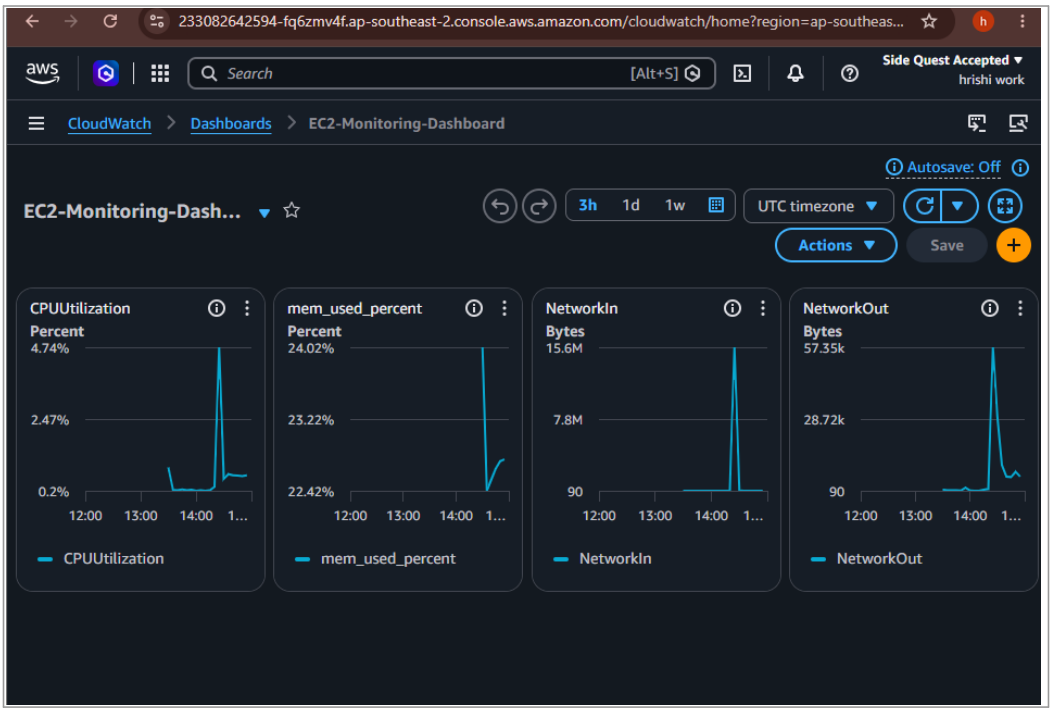
Loaded the config with fetch-config. Status: running, configured.



CWAgent metrics mem_used_percent and swap_used_percent for MyWeb-app.

Task 4: Create a CloudWatch Dashboard

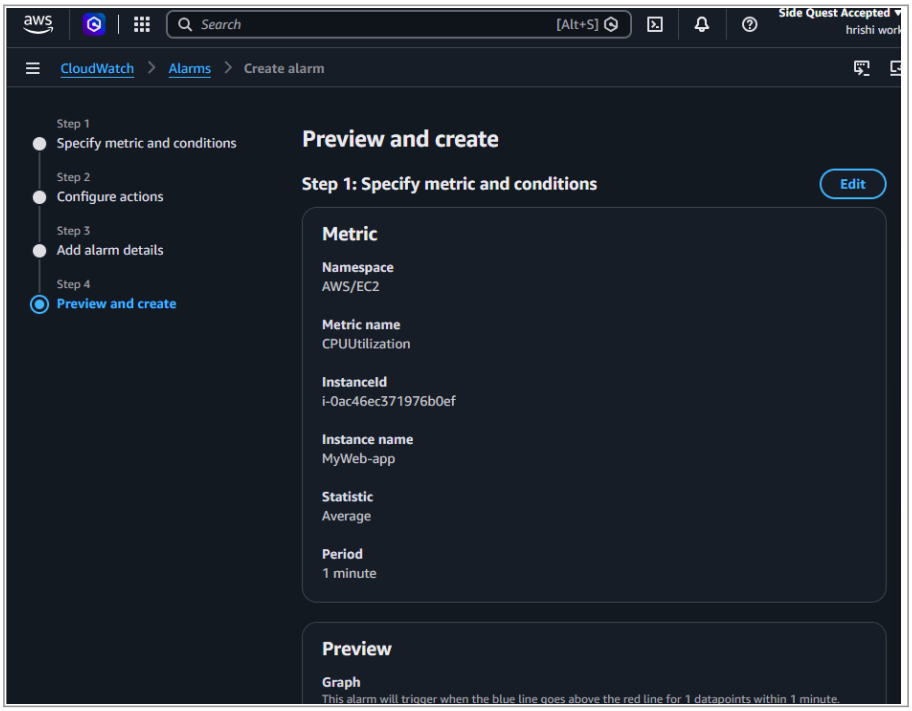
- Created a CloudWatch dashboard showing CPU, memory and network metrics for the EC2 instance.
- All widgets are live and show the activity spike around 14:00 UTC.



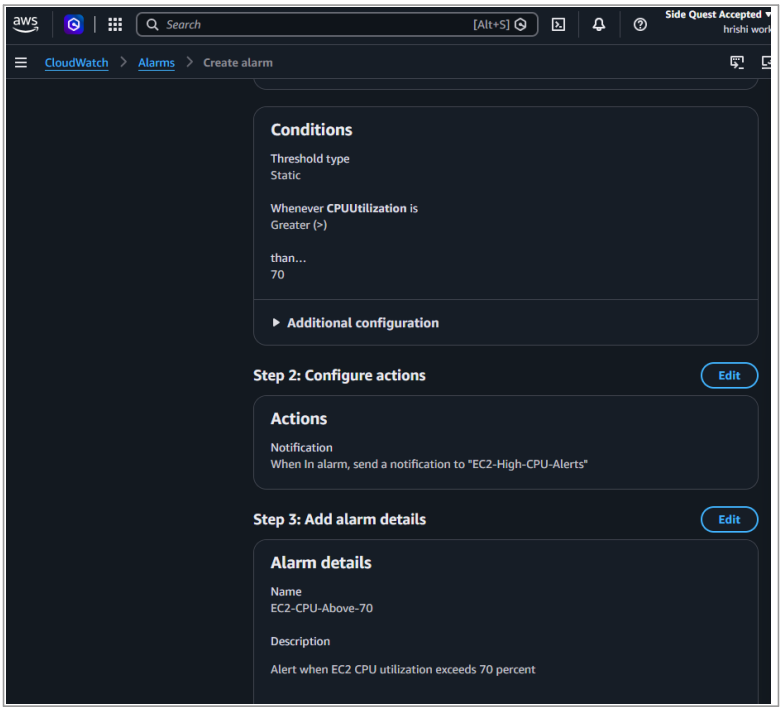
Dashboard EC2-Monitoring-Dashboard with CPUUtilization, mem_used_percent, NetworkIn and NetworkOut.

Task 5: Create a CloudWatch Alarm

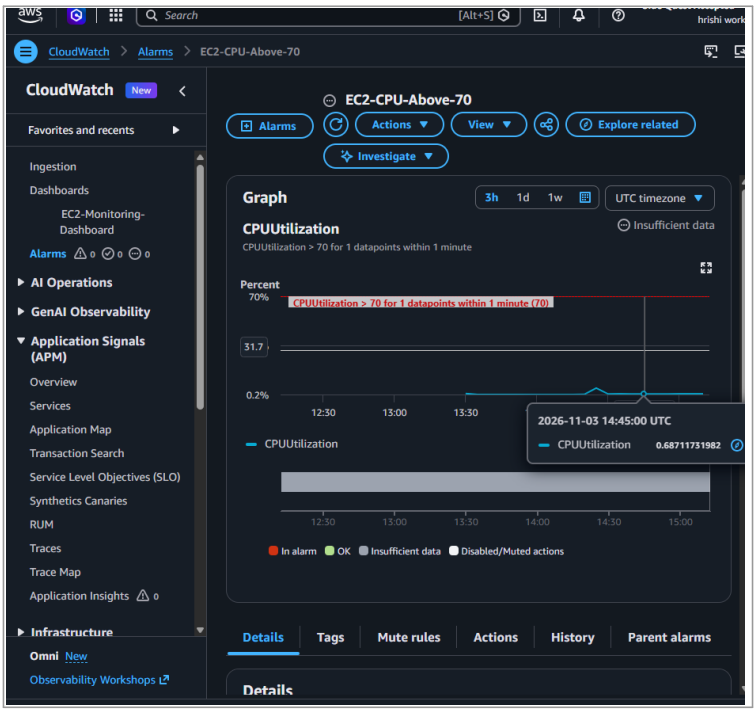
- Created the alarm EC2-CPU-Above-70, which triggers when CPU utilization goes above 70%.
- Configured it to send a notification to the SNS topic EC2-High-CPU-Alerts when in alarm.



Selected the CPUUtilization metric (Average, 1-minute period) for MyWeb-app.



Threshold set to greater than 70, with SNS topic EC2-High-CPU-Alerts for notifications.



Alarm EC2-CPU-Above-70 created (waiting for data).

Task 6: Generate High CPU Utilization

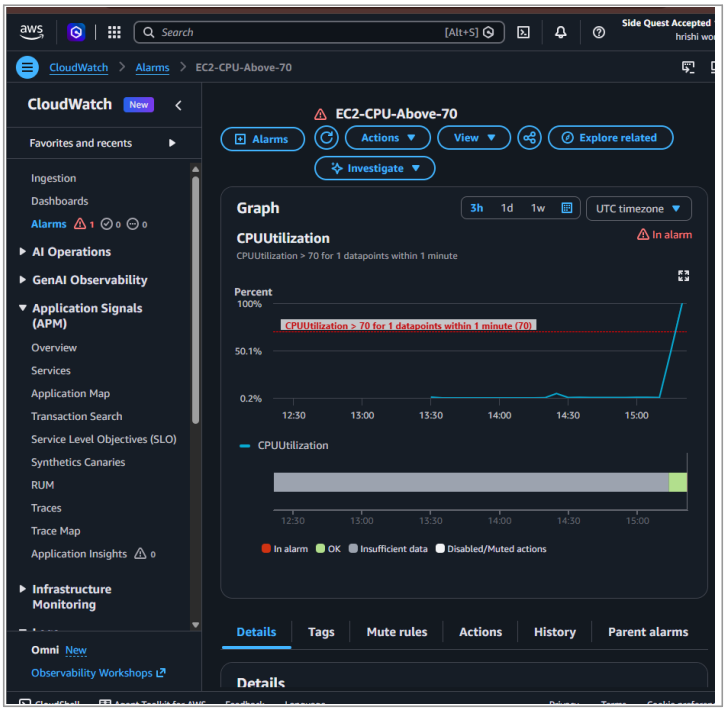
- Generated CPU load with stress-ng and observed the alarm change from OK to ALARM.
- Email notifications are delivered through the SNS topic EC2-High-CPU-Alerts.

```
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :
  Installing       : lksctp-tools-1.0.18-9.amzn2023.0.3.x86_64
  Installing       : libbsd-0.10.0-7.amzn2023.0.2.x86_64
  Installing       : Judy-1.0.5-25.amzn2023.0.3.x86_64
  Installing       : stress-ng-0.15.05-1.amzn2023.x86_64
  Running scriptlet: stress-ng-0.15.05-1.amzn2023.x86_64
  Verifying        : Judy-1.0.5-25.amzn2023.0.3.x86_64
  Verifying        : libbsd-0.10.0-7.amzn2023.0.2.x86_64
  Verifying        : lksctp-tools-1.0.18-9.amzn2023.0.3.x86_64
  Verifying        : stress-ng-0.15.05-1.amzn2023.x86_64

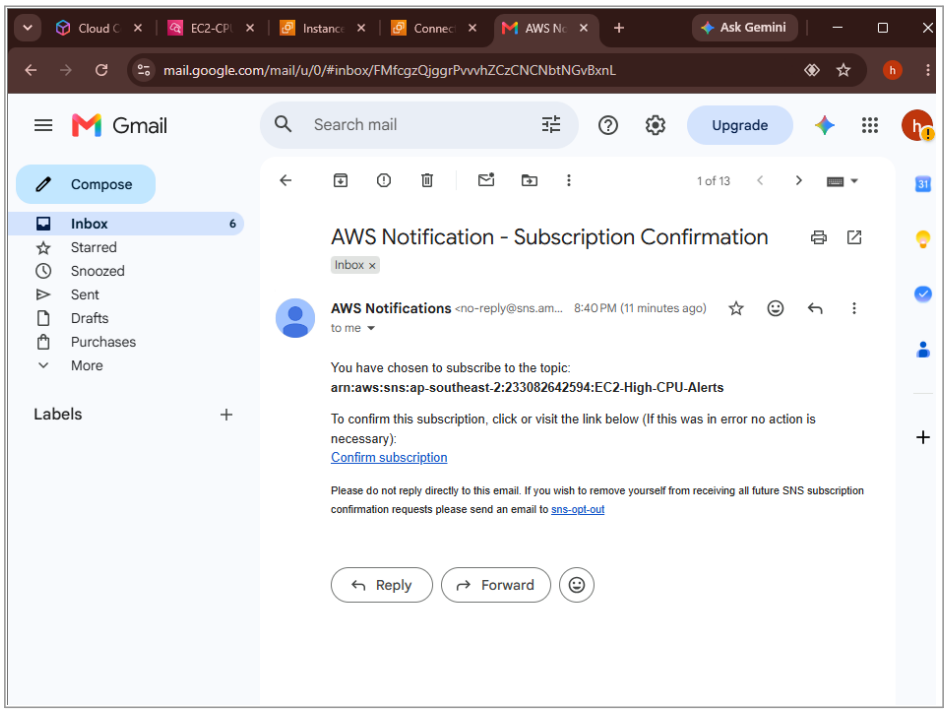
Installed:
  Judy-1.0.5-25.amzn2023.0.3.x86_64    libbsd-0.10.0-7.amzn2023.0.2.x86_64  lksctp-tools-1.0.18-9.amzn2023.0.3.x86_
  stress-ng-0.15.05-1.amzn2023.x86_64

Complete!
[ec2-user@ip-172-31-37-254 ~]$ stress-ng --version
stress-ng, version 0.15.05 (gcc 11.3.1, x86_64 Linux 6.18.56-125.164.amzn2023.x86_64) 💻🔥
[ec2-user@ip-172-31-37-254 ~]$ nproc
2
[ec2-user@ip-172-31-37-254 ~]$ stress-ng --cpu 2 --timeout 300s
stress-ng: info:  [29334] setting to a 300 second (5 mins, 0.00 secs) run per stressor
stress-ng: info:  [29334] dispatching hogs: 2 cpu
```

Installed stress-ng and ran: stress-ng --cpu 2 --timeout 300s.



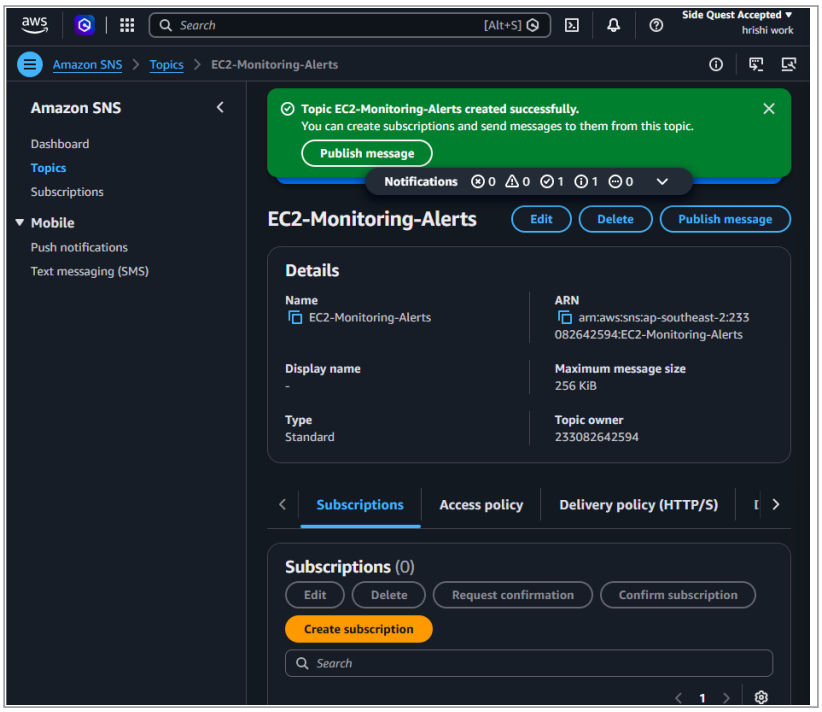
Alarm EC2-CPU-Above-70 moved from OK to In alarm as CPU reached 100%.



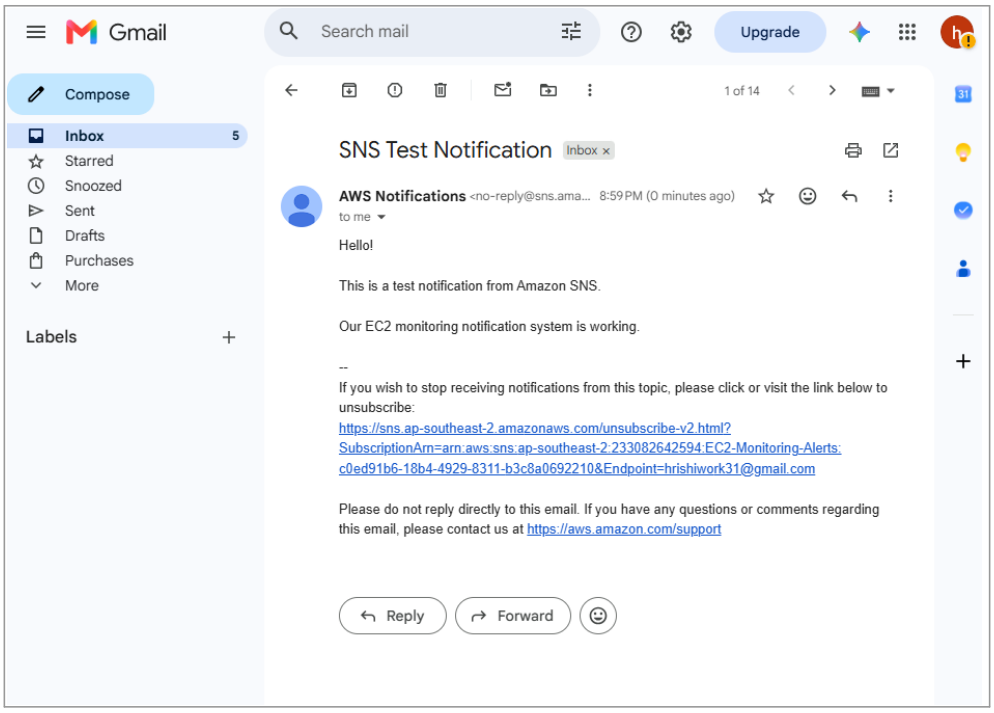
AWS Notification email for the SNS topic EC2-High-CPU-Alerts.

Task 7: Configure Amazon SNS Notifications

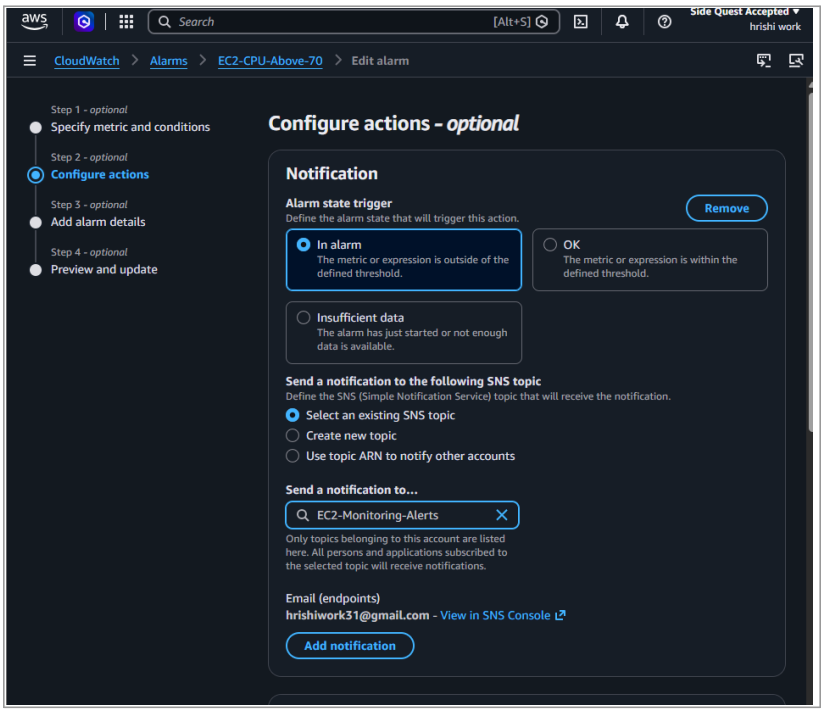
- Created an SNS topic, subscribed an email address and verified delivery with a test message.
- Integrated the topic with the CloudWatch alarm so it notifies the subscribed email when the alarm goes into ALARM.



Created the SNS topic EC2-Monitoring-Alerts (Standard).



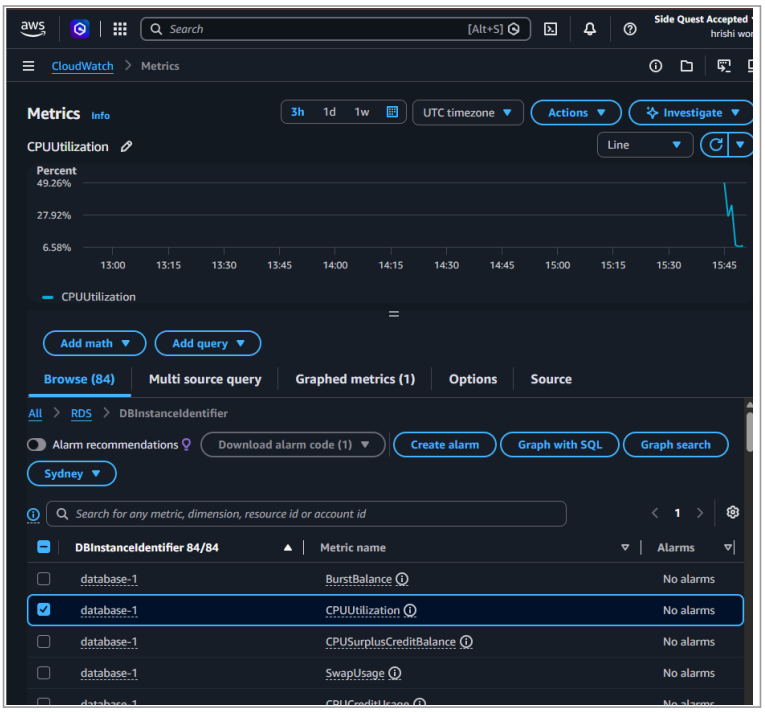
Subscribed email received the SNS test notification.



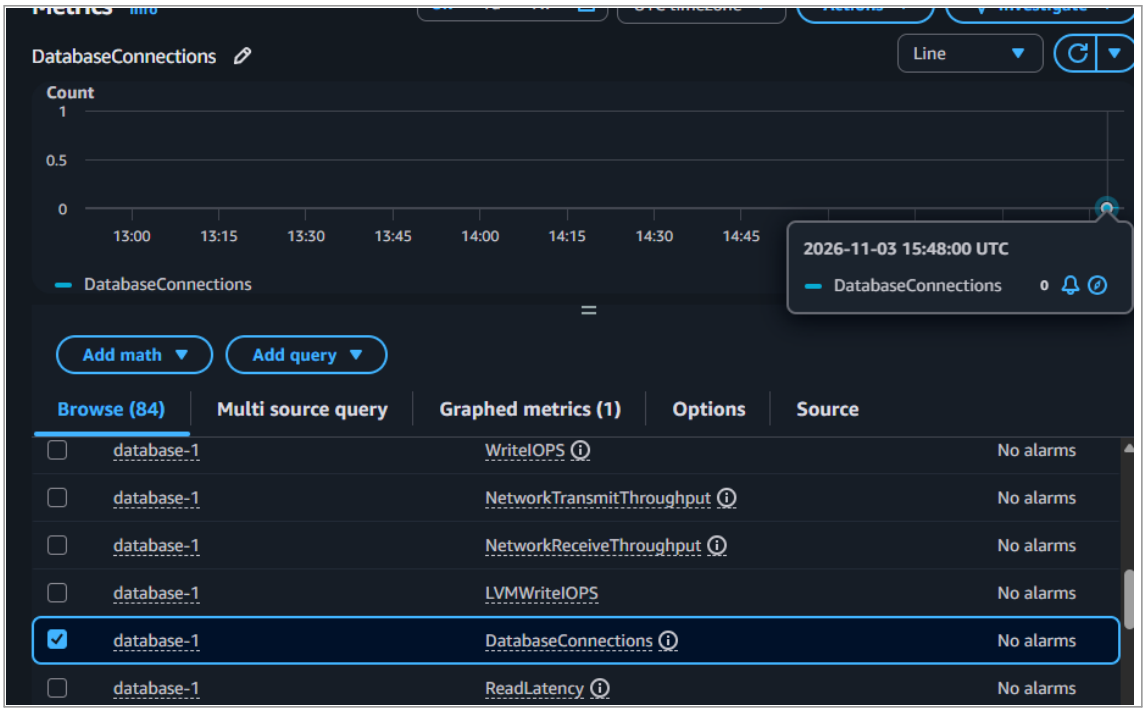
Alarm EC2-CPU-Above-70 set to send an In alarm notification to EC2-Monitoring-Alerts.

Task 8: Monitor an Amazon RDS Database

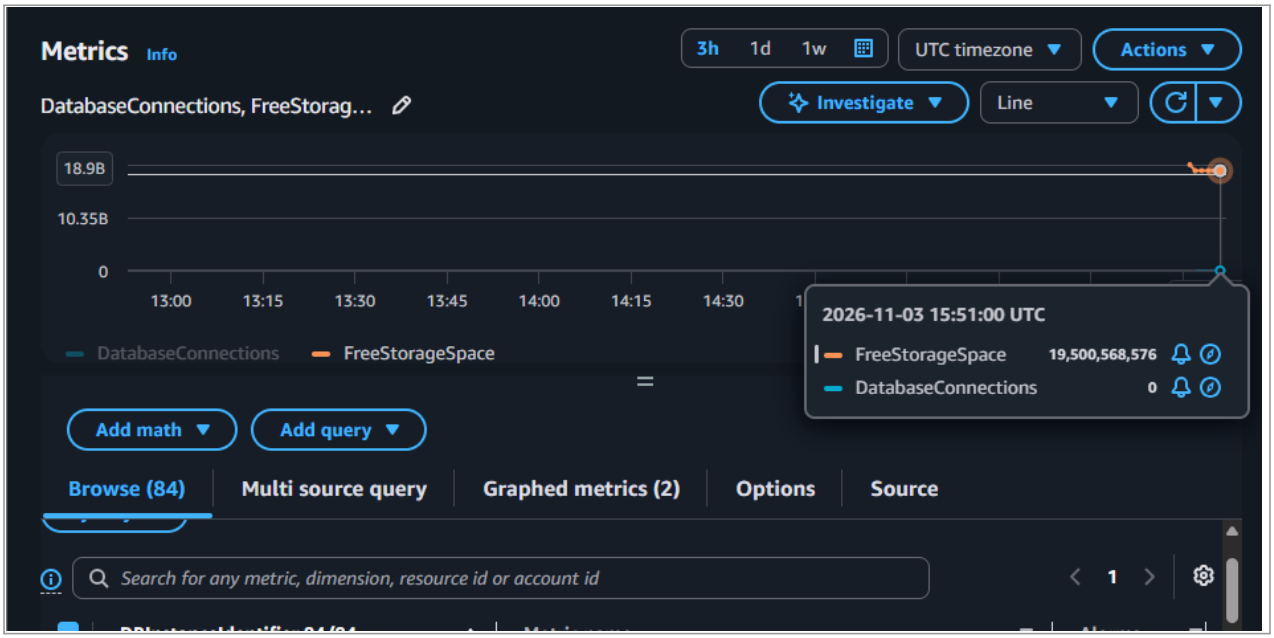
- Monitored the RDS instance database-1 in CloudWatch Metrics: CPU Utilization, Database Connections and Free Storage Space.
- The database is idle, with 0 connections and plenty of free storage.



RDS metrics for database-1, graphing CPUUtilization.



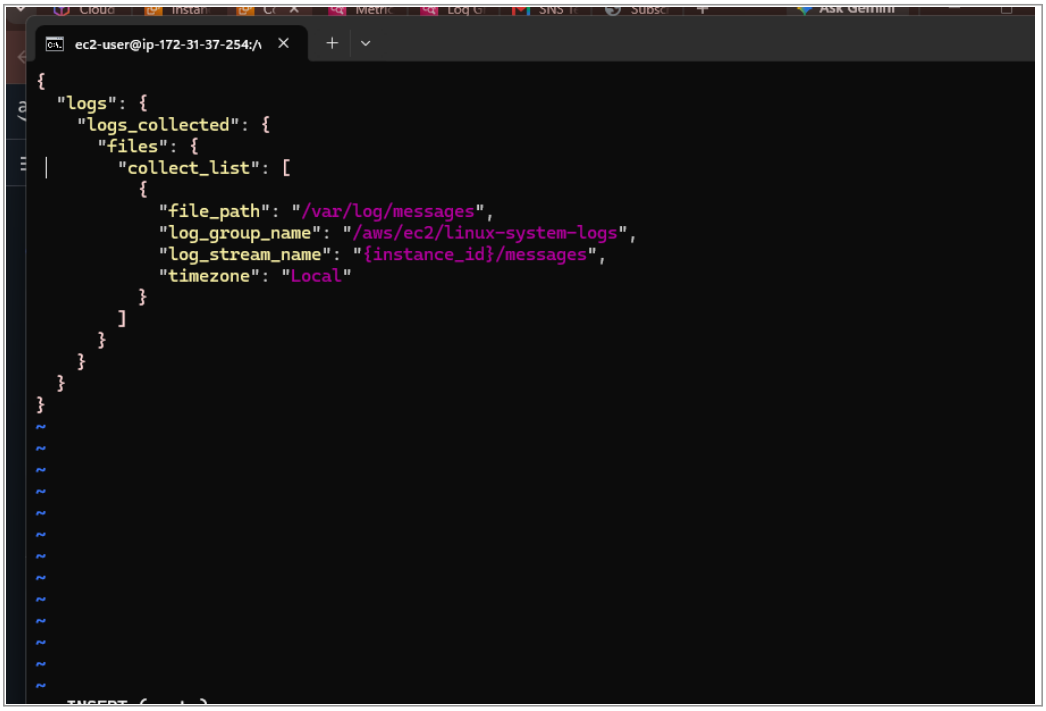
DatabaseConnections graph showing 0 active connections.



FreeStorageSpace added to the graph (about 19.5 GB free).

Task 9: Explore CloudWatch Logs

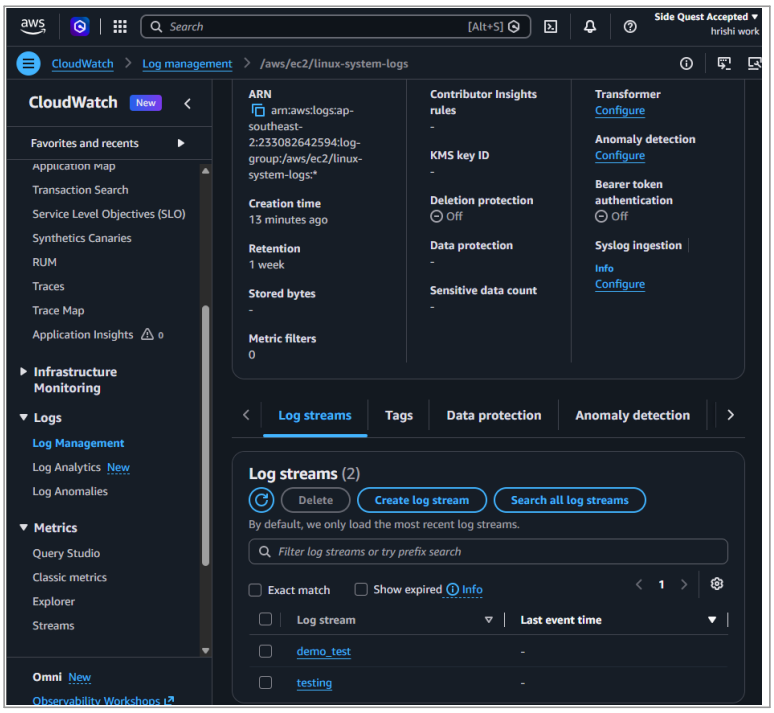
- Created the log group /aws/ec2/linux-system-logs and configured the CloudWatch Agent to send system logs (/var/log/messages) to it.
- Restarted the agent with the new config and confirmed the log group in the CloudWatch console.



Agent config to collect /var/log/messages into the log group /aws/ec2/linux-system-logs.

```
Start configuration validation...
2026/11/03 15:59:00 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json ...
2026/11/03 15:59:00 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_amazon-cloudwatch-agent.json ...
2026/11/03 15:59:00 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_logs-config.json.tmp ...
2026/11/03 15:59:01 I! Valid Json input schema.
2026/11/03 15:59:01 D! ec2tagger processor required because append_dimensions is set
2026/11/03 15:59:01 Configuration validation first phase succeeded
Starting config-translator, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-translator -input /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -input-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -output /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -mode ec2 -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config append]/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json does not exist or cannot read. Skipping it.
I! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml
Configuration validation second phase succeeded
Configuration validation succeeded
[ec2-user@ip-172-31-37-254 log]$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
{
  "status": "running",
  "starttime": "2026-11-03T15:59:01+00:00",
  "configstatus": "configured",
  "version": "1.300073.2"
}
[ec2-user@ip-172-31-37-254 log]$
```

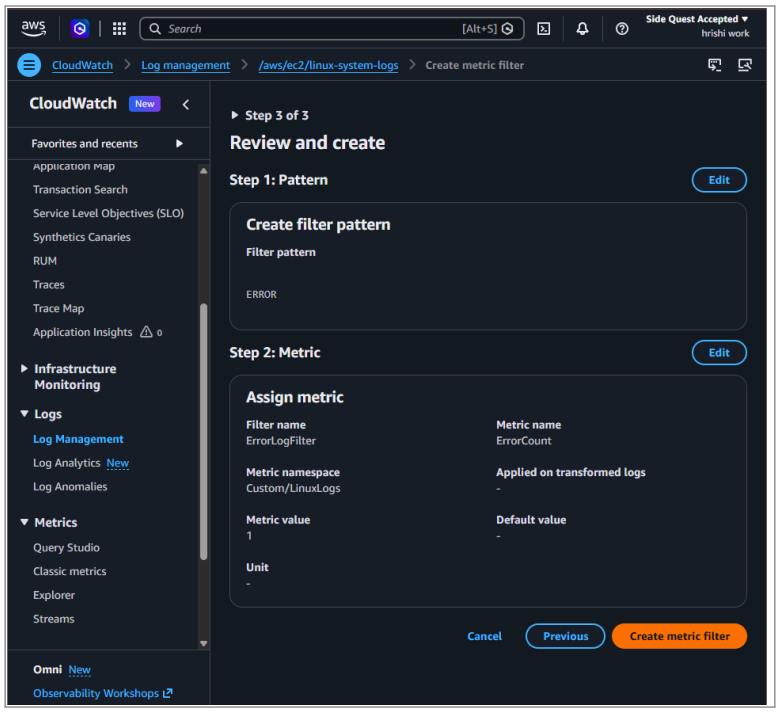
Loaded the logs config with fetch-config. Status: running, configured.



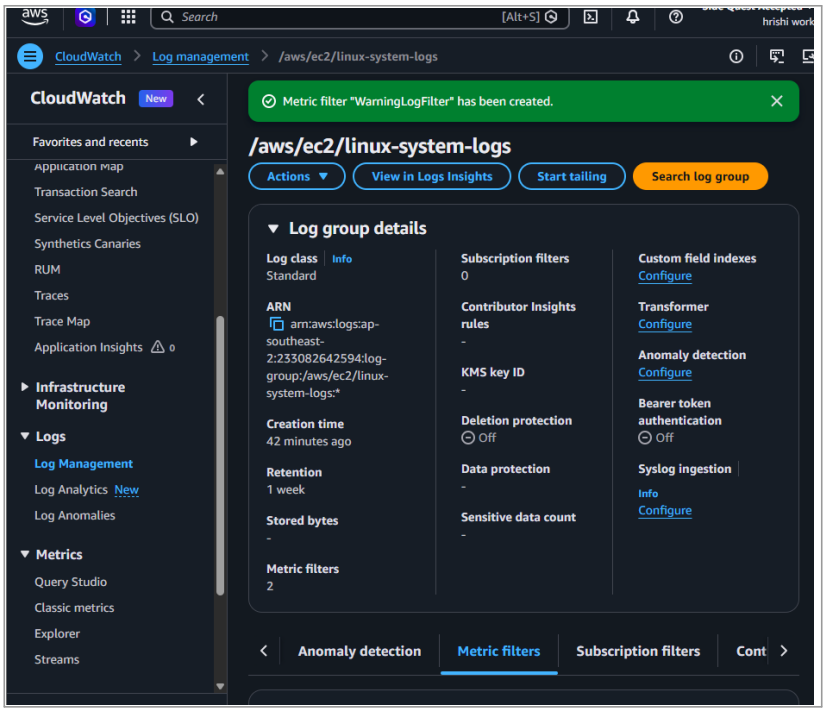
Log group /aws/ec2/linux-system-logs in CloudWatch Logs.

Task 10: Create a Metric Filter

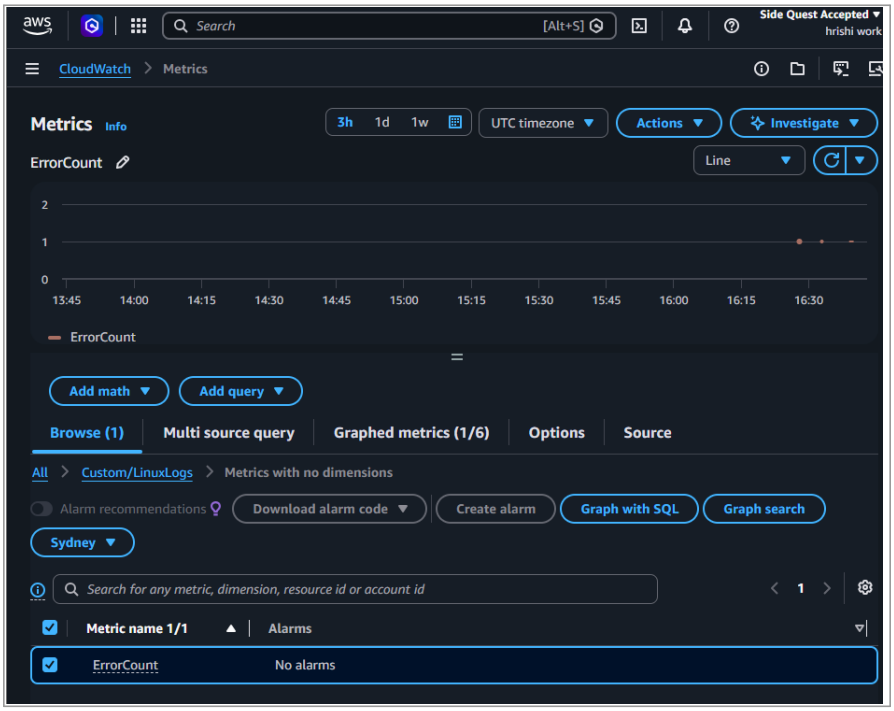
- Created metric filters on the log group to count ERROR and WARNING log entries.
- Confirmed that the custom metric ErrorCount is generated in the Custom/LinuxLogs namespace.



Metric filter ErrorLogFilter (pattern ERROR) publishing ErrorCount to Custom/LinuxLogs.



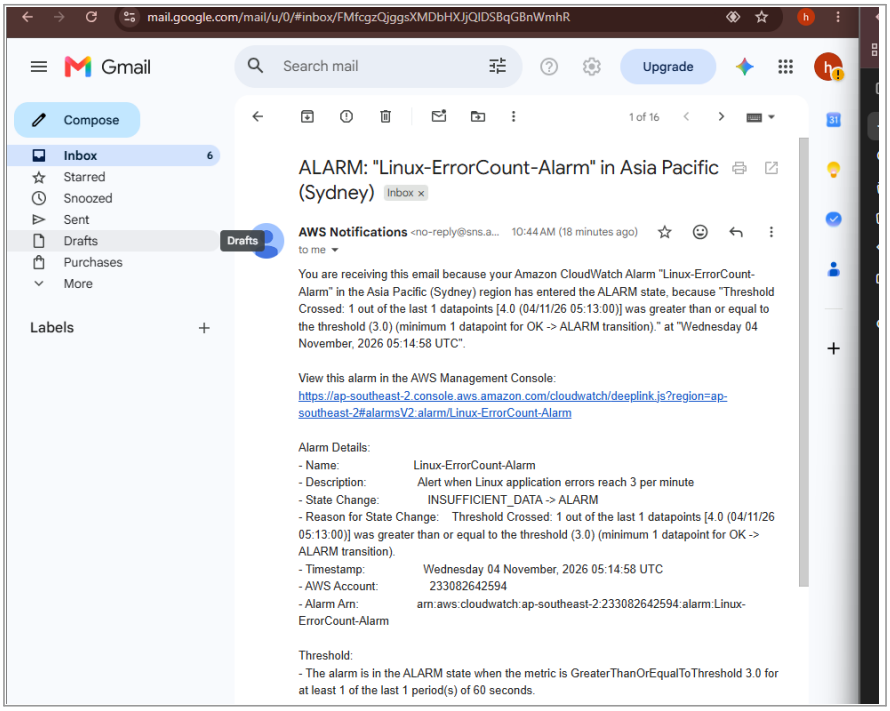
Second filter WarningLogFilter created. The log group now has 2 metric filters.



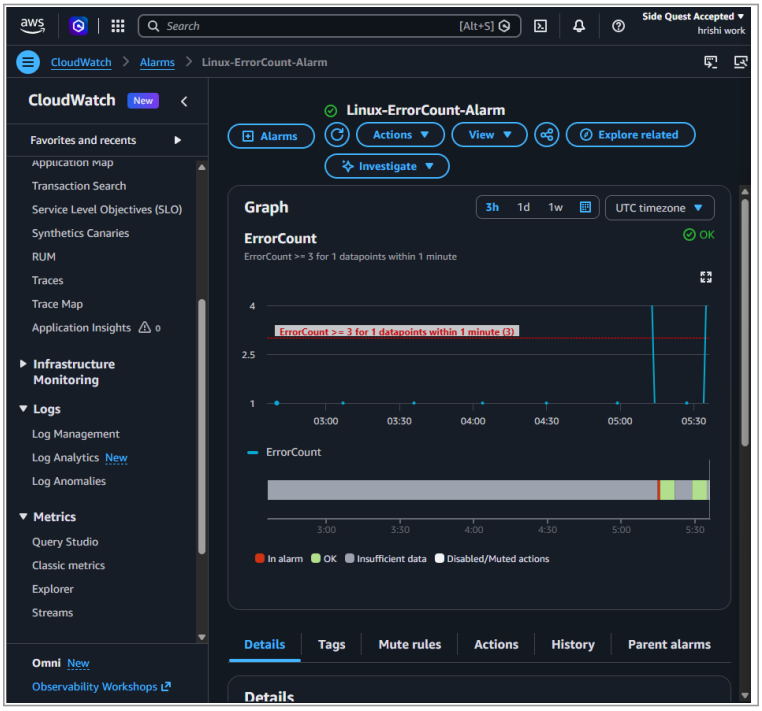
Custom metric ErrorCount in CloudWatch Metrics.

Task 11: Create an Alarm Using Log Metrics

- Created the alarm Linux-ErrorCount-Alarm, which triggers when ErrorCount is greater than or equal to 3 within 1 minute.
- Confirmed the notification: SNS sent an email to the subscribed address when the threshold was crossed.



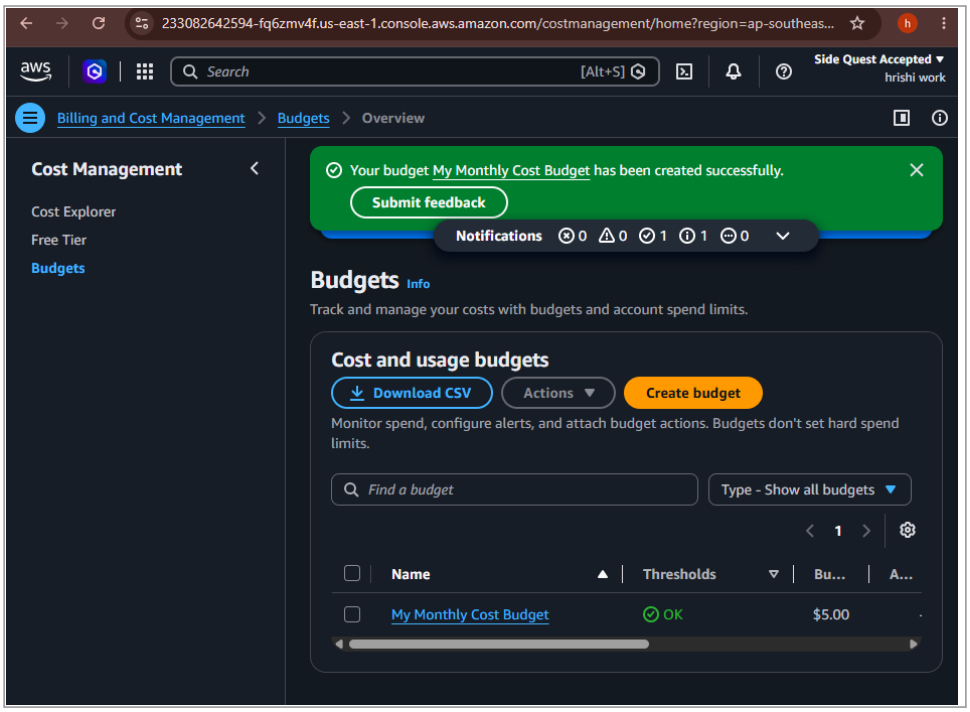
SNS email ALARM: Linux-ErrorCount-Alarm (ErrorCount reached 4, threshold 3).



Alarm graph showing the spike above the threshold and the state back to OK.

Task 12: Create an AWS Billing Alarm

- Enabled billing monitoring and created a budget to track estimated AWS charges.
- The budget My Monthly Cost Budget ($5.00) was created successfully and its threshold status is OK.



Budget My Monthly Cost Budget created successfully ($5.00).